

Finding and evaluating community structure in networks

M. E. J. Newman^{1,2} and M. Girvan^{2,3}

¹*Department of Physics and Center for the Study of Complex Systems,
University of Michigan, Ann Arbor, MI 48109-1120*

²*Santa Fe Institute, 1399 Hyde Park Road, Santa Fe, NM 87501*

³*Department of Physics, Cornell University, Ithaca, NY 14853-2501*

We propose and study a set of algorithms for discovering community structure in networks—natural divisions of network nodes into densely connected subgroups. Our algorithms all share two definitive features: first, they involve iterative removal of edges from the network to split it into communities, the edges removed being identified using one of a number of possible “betweenness” measures, and second, these measures are, crucially, recalculated after each removal. We also propose a measure for the strength of the community structure found by our algorithms, which gives us an objective metric for choosing the number of communities into which a network should be divided. We demonstrate that our algorithms are highly effective at discovering community structure in both computer-generated and real-world network data, and show how they can be used to shed light on the sometimes dauntingly complex structure of networked systems.

I. INTRODUCTION

Empirical studies and theoretical modeling of networks have been the subject of a large body of recent research in statistical physics and applied mathematics [1, 2, 3, 4]. Network ideas have been applied with great success to topics as diverse as the Internet and the world wide web [5, 6, 7], epidemiology [8, 9, 10, 11], scientific citation and collaboration [12, 13], metabolism [14, 15], and ecosystems [16, 17], to name but a few. A property that seems to be common to many networks is *community structure*, the division of network nodes into groups within which the network connections are dense, but between which they are sparser—see Fig. 1. The ability to find and analyze such groups can provide invaluable help in understanding and visualizing the structure of networks. In this paper we show how this can be achieved.

The study of community structure in networks has a long history. It is closely related to the ideas of graph partitioning in graph theory and computer science, and

hierarchical clustering in sociology [18, 19]. Before presenting our own findings, it is worth reviewing some of this preceding work, to understand its achievements and where it falls short.

Graph partitioning is a problem that arises in, for example, parallel computing. Suppose we have a number n of intercommunicating computer processes, which we wish to distribute over a number g of computer processors. Processes do not necessarily need to communicate with all others, and the pattern of required communications can be represented by a graph or network in which the vertices represent processes and edges join process pairs that need to communicate. The problem is to allocate the processes to processors in such a way as roughly to balance the load on each processor, while at the same time minimizing the number of edges that run between processors, so that the amount of interprocessor communication (which is normally slow) is minimized. In general, finding an exact solution to a partitioning task of this kind is believed to be an NP-complete problem, making it prohibitively difficult to solve for large graphs, but a wide variety of heuristic algorithms have been developed that give acceptably good solutions in many cases, the best known being perhaps the Kernighan–Lin algorithm [20], which runs in time $O(n^3)$ on sparse graphs.

A solution to the graph partitioning problem is however not particularly helpful for analyzing and understanding networks in general. If we merely want to find if and how a given network breaks down into communities, we probably don’t know how many such communities there are going to be, and there is no reason why they should be roughly the same size. Furthermore, the number of inter-community edges needn’t be strictly minimized either, since more such edges are admissible between large communities than between small ones.

As far as our goals in this paper are concerned, a more useful approach is that taken by social network analysis with the set of techniques known as hierarchical clustering. These techniques are aimed at discovering natural divisions of (social) networks into groups, based on var-

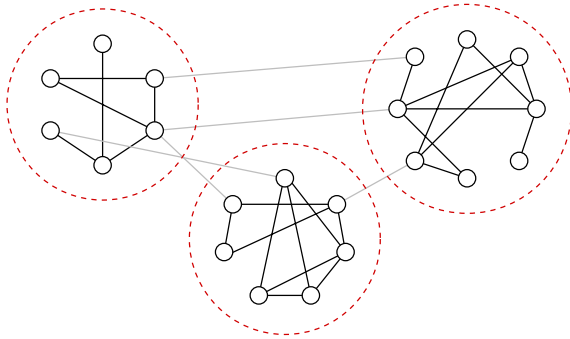


FIG. 1: A small network with community structure of the type considered in this paper. In this case there are three communities, denoted by the dashed circles, which have dense internal links but between which there are only a lower density of external links.

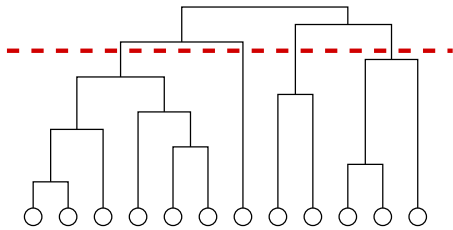


FIG. 2: A hierarchical tree or dendrogram illustrating the type of output generated by the algorithms described here. The circles at the bottom of the figure represent the individual vertices of the network. As we move up the tree the vertices join together to form larger and larger communities, as indicated by the lines, until we reach the top, where all are joined together in a single community. Alternatively, we the dendrogram depicts an initially connected network splitting into smaller and smaller communities as we go from top to bottom. A cross-section of the tree at any level, as indicated by the dotted line, will give the communities at that level. The vertical height of the split-points in the tree are indicative only of the order in which the splits (or joins) took place, although it is possible to construct more elaborate dendrograms in which these heights contain other information.

ious metrics of similarity or strength of connection between vertices. They fall into two broad classes, agglomerative and divisive [19], depending on whether they focus on the addition or removal of edges to or from the network. In an agglomerative method, similarities are calculated by one method or another between vertex pairs, and edges are then added to an initially empty network (n vertices with no edges) starting with the vertex pairs with highest similarity. The procedure can be halted at any point, and the resulting components in the network are taken to be the communities. Alternatively, the entire progression of the algorithm from empty graph to complete graph can be represented in the form of a tree or *dendrogram* such as that shown in Fig. 2. Horizontal cuts through the tree represent the communities appropriate to different halting points.

Agglomerative methods based on a wide variety of similarity measures have been applied to different networks. Some networks have natural similarity metrics built in. For example, in the widely studied network of collaborations between film actors [21, 22], in which two actors are connected if they have appeared in the same film, one could quantify similarity by how many films actors have appeared in together [23]. Other networks have no natural metric, but suitable ones can be devised using correlation coefficients, path lengths, or matrix methods. A well known example of an agglomerative clustering method is the Concor algorithm of Breiger *et al.* [24].

Agglomerative methods have their problems however. One concern is that they fail with some frequency to find the correct communities in networks where the community structure is known, which makes it difficult to place much trust in them in other cases. Another is their ten-

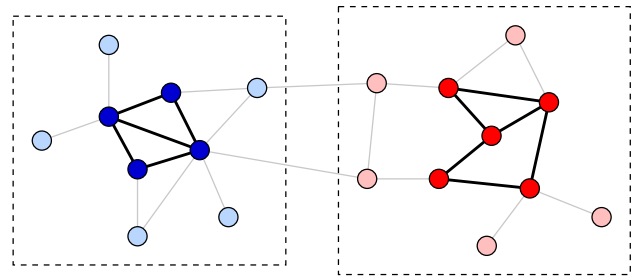


FIG. 3: Agglomerative clustering methods are typically good at discovering the strongly linked cores of communities (bold vertices and edges) but tend to leave out peripheral vertices, even when, as here, most of them clearly belong to one community or another.

dency to find only the cores of communities and leave out the periphery. The core nodes in a community often have strong similarity, and hence are connected early in the agglomerative process, but peripheral nodes that have no strong similarity to others tend to get neglected, leading to structures like that shown in Fig. 3. In this figure, there are a number of peripheral nodes whose community membership is obvious to the eye—in most cases they have only a single link to a specific community—but agglomerative methods often fail to place such nodes correctly.

In this paper, therefore, we focus on divisive methods. These methods have been relatively little studied in the previous literature, either in social network theory or elsewhere, but, as we will see, seem to offer a lot of promise. In a divisive method, we start with the network of interest and attempt to find the *least* similar connected pairs of vertices and then remove the edges between them. By doing this repeatedly, we divide the network into smaller and smaller components, and again we can stop the process at any stage and take the components at that stage to be the network communities. Again, the process can be represented as a dendrogram depicting the successive splits of the network into smaller and smaller groups.

The approach we take follows roughly these lines, but adopts a somewhat different philosophical viewpoint. Rather than looking for the most weakly connected vertex pairs, our approach will be to look for the edges in the network that are most “between” other vertices, meaning that the edge is, in some sense, responsible for connecting many pairs of others. Such edges need not be weak at all in the similarity sense. How this idea works out in practice will become clear in the course of the presentation.

Briefly then, the outline of this paper is as follows. In Sec. II we describe the crucial concepts behind our methods for finding community structure in networks and show how these concepts can be turned into a concrete prescription for performing calculations. In Sec. III we describe in detail the implementation of our methods. In

Sec. IV we consider ways of determining when a particular division of a network into communities is a good one, allowing us to quantify the success of our community-finding algorithms. And in Sec. V we give a number of applications of our algorithms to particular networks, both real and artificial. In Sec. VI we give our conclusions. A brief report of some of the work contained in this paper has appeared previously as Ref. 25.

II. FINDING COMMUNITIES IN A NETWORK

In this paper we present a class of new algorithms for network clustering, i.e., the discovery of community structure in networks. Our discussion focuses primarily on networks with only a single type of vertex and a single type of undirected, unweighted edge, although generalizations to more complicated network types are certainly possible.

There are two central features that distinguish our algorithms from those that have preceded them. First, our algorithms are divisive rather than agglomerative. Divisive algorithms have occasionally been studied in the past, but, as discussed in the introduction, ours differ in focusing not on removing the edges between vertex pairs with lowest similarity, but on finding edges with the highest “betweenness,” where betweenness is some measure that favors edges that lie between communities and disfavors those that lie inside communities.

To make things more concrete, we give some examples of the types of betweenness measures we will be looking at. All of them are based on the same idea. If two communities are joined by only a few inter-community edges, then all paths through the network from vertices in one community to vertices in the other must pass along one of those few edges. Given a suitable set of paths, one can count how many go along each edge in the graph, and this number we then expect to be largest for the inter-community edges, thus providing a method for identifying them. Our different measures correspond to various implementations of this idea.

1. The simplest example of such a betweenness measure is that based on shortest (geodesic) paths: we find the shortest paths between all pairs of vertices and count how many run along each edge. To the best of our knowledge this measure was first introduced by Anthonisse in a never-published technical report in 1971 [26]. Anthonisse called it “rush,” but we prefer the term *edge betweenness*, since the quantity is a natural generalization to edges of the well-known (vertex) betweenness measure of Freeman [27], which was the inspiration for our approach. When we need to distinguish it from the other betweenness measures considered in this paper, we will refer to it as *shortest-path betweenness*. A fast algorithm for calculating the shortest-path betweenness is given in Sec. III A.

2. The shortest-path betweenness can be thought of in terms of signals traveling through a network. If signals travel from source to destination along geodesic network paths, and all vertices send signals at the same constant rate to all others, then the betweenness is a measure of the rate at which signals pass along each edge. Suppose however that signals do not travel along geodesic paths, but instead just perform a random walk about the network until they reach their destination. This gives us another measure on edges, the *random-walk betweenness*: we calculate the expected net number of times that a random walk between a particular pair of vertices will pass down a particular edge and sum over all vertex pairs. The random-walk betweenness can be calculated using matrix methods, as described in Sec. III C.

3. Another betweenness measure is motivated by ideas from elementary circuit theory. We consider the circuit created by placing a unit resistance on each edge of the network and unit current source and sink at a particular pair of vertices. The resulting current flow in the network will travel from source to sink along a multitude of paths, those with least resistance carrying the greatest fraction of the current. The *current-flow betweenness* for an edge we define to be the absolute value of the current along the edge summed over all source/sink pairs. It can be calculated using Kirchhoff’s laws, as described in Sec. III B. In fact, as we will show, the current-flow betweenness turns out to be exactly the same as the random walk betweenness of the previous paragraph, but we nonetheless consider it separately since it leads to a simpler derivation of the measure.

These measures are only suggestions; many others are possible and may well be appropriate for specific applications. Measures (1) and (2) are in some sense extremes in the spectrum of possibilities, one corresponding to signals that know exactly where they are going, and the other to signals that have no idea where they are going. As we will see, however, these two measures actually give rather similar results, indicating that the precise choice of betweenness measure may not, at least for the types of applications considered here, be that important.

The second way in which our methods differ from previous ones is in the inclusion of a “recalculation step” in the algorithm. If we were to perform a standard divisive clustering based on edge betweenness we would calculate the edge betweenness for all edges in the network and then remove edges in decreasing order of betweenness to produce a dendrogram like that of Fig. 2, showing the order in which the network split up.

However, once the first edge in the network is removed in such an algorithm, the betweenness values for the remaining edges will no longer reflect the network as it now is. This can give rise to unwanted behaviors. For exam-

ple, if two communities are joined by two edges, but, for one reason or another, most paths between the two flow along just one of those edges, then that edge will have a high betweenness score and the other will not. An algorithm that calculated betweennesses only once and then removed edges in betweenness order would remove the first edge early in the course of its operation, but the second might not get removed until much later. Thus the obvious division of the network into two parts might not be discovered by the algorithm. In the worst case the two parts themselves might be individually broken up before the division between the two is made. In practice, problems like this crop up in real networks with some regularity and render algorithms of this type ineffective for the discovery of community structure.

The solution, luckily, is obvious. We simply recalculate our betweenness measure after the removal of each edge. This certainly adds to the computational effort of performing the calculation, but its effect on the results is so desirable that we consider the price worth paying.

Thus the general form of our community structure finding algorithm is as follows:

1. Calculate betweenness scores for all edges in the network.
2. Find the edge with the highest score and remove it from the network.
3. Recalculate betweenness for all remaining edges.
4. Repeat from step 2.

In fact, it appears that the recalculation step is the most important feature of the algorithm, as far as getting satisfactory results is concerned. As mentioned above, our studies indicate that, once one hits on the idea of using betweenness measures to weight edges, the exact measure one uses appears not to influence the results highly. The recalculation step, on the other hand, is absolutely crucial to the operation of our methods. This step was missing from previous attempts at solving the clustering problem using divisive algorithms, and yet without it the results are very poor indeed, failing to find known community structure even in the simplest of cases. In Sec. VB we give an example comparing the performance of the algorithm on a particular network with and without the recalculation step.

In the following sections we discuss implementation and give examples of our algorithms for finding community structure. For the reader who merely wants to know what algorithm they should use for their own problem, let us give an immediate answer: for most problems, we recommend the algorithm with betweenness scores calculated using the shortest-path betweenness measure (1) above. This measure appears to work well and is the quickest to calculate—as described in Sec. III A, it can be calculated for all edges in time $O(mn)$, where m is the number of edges in the graph and n is the number of vertices. This is the only version of the algorithm

that we discussed in Ref. 25 [47]. The other versions we discuss, while being of some pedagogical interest, make greater computational demands, and in practice seem to give results no better than the shortest-path method.

III. IMPLEMENTATION

In theory, the descriptions of the last section completely define the methods we consider in this paper, but in practice there are a number of tricks to their implementation that are important for turning the description into a workable computer algorithm.

Essentially all of the work in the algorithm is in the calculation of the betweenness scores for the edges; the job of finding and removing the highest-scoring edge is trivial and not computationally demanding. Let us tackle our three suggested betweenness measures in turn.

A. Shortest-path betweenness

At first sight, it appears that calculating the edge betweenness measure based on geodesic paths for all edges will take $O(mn^2)$ operations on a graph with m edges and n vertices: calculating the shortest path between a particular pair of vertices can be done using breadth-first search in time $O(m)$ [28, 29], and there are $O(n^2)$ vertex pairs. Recently however new algorithms have been proposed by Newman [30] and independently by Brandes [31] that can perform the calculation faster than this, finding all betweennesses in $O(mn)$ time. Both Newman and Brandes gave algorithms for the standard Freeman vertex betweenness, but it is trivial to adapt their algorithms for edge betweenness. We describe the resulting method here for the algorithm of Newman.

Breadth-first search can find shortest paths from a single vertex s to all others in time $O(m)$. In the simplest case, when there is only a single shortest path from the source vertex to any other (we will consider other cases in a moment) the resulting set of paths forms a shortest-path tree—see Fig. 4a. We can now use this tree to calculate the contribution to betweenness for each edge from this set of paths as follows. We find first the “leaves” of the tree, i.e., those nodes such that no shortest paths to other nodes pass through them, and we assign a score of 1 to the single edge that connects each to the rest of the tree, as shown in the figure. Then, starting with those edges that are farthest from the source vertex on the tree, i.e., lowest in Fig. 4a, we work upwards, assigning a score to each edge that is 1 plus the sum of the scores on the neighboring edges immediately below it. When we have gone through all edges in the tree, the resulting scores are the betweenness counts for the paths from vertex s . Repeating the process for all possible vertices s and summing the scores, we arrive at the full betweenness scores for shortest paths between all pairs. The breadth-first search and the process of working up through the tree

to about $n = 10\,000$ vertices, with current (*circa* 2003) desktop computers. In some special cases one can do better. In particular, we note that the removal of an edge only affects the betweenness of other edges that fall in the same component, and hence that we need only recalculate betweennesses in that component. Networks with strong community structure often break apart into separate components quite early in the progress of the algorithm, substantially reducing the amount of work that needs to be done on subsequent steps. Whether this results in a change in the computational complexity of the algorithm for any commonly occurring classes of graphs is an open question, but it certainly gives a substantial speed boost to many of the calculations described in this paper.

Some networks are directed, i.e., their edges run in one direction only. The world wide web is an example; links in the web point in one direction only from one web page to another. One could imagine a generalization of the shortest-path betweenness that allowed for directed edges by counting only those paths that travel in the forward direction along edges. Such a calculation is a trivial variation on the one described above. However, we have found that in many cases it is better to ignore the directed nature of a network in calculating community structure. Often an edge acts simply as an indication of a connection between two nodes, and its direction is unimportant. For example, in Ref. 25 we applied our algorithm to a food web of predator-prey interactions between marine species. Predator-prey interactions are clearly directed—one species may eat another, but it is unlikely that the reverse is simultaneously true. However, as far as community structure is concerned, we want to know only which species have interactions with which others. We find, therefore, that our algorithm applied to the undirected version of the food web works well at picking out the community structure, and no special algorithm is needed for the directed case. We give another example of our method applied to a directed graph in Sec. VD.

B. Resistor networks

As examples of betweenness measures that take more than just shortest paths into account, we proposed in Sec. II measures based on random walks and resistor networks. In fact, as we now show, when appropriately defined these two measures are precisely the same. Here we derive the resistance measure first, since it turns out to be simpler; in the following section we derive the random walk measure and show that the two are equivalent.

Consider the network created by placing a unit resistance on every edge of our network, a unit current source at vertex s , and a unit current sink at vertex t (see Fig. 5). Clearly the current between s and t will flow primarily along short paths, but some will flow along longer ones, roughly in inverse proportion to their length. We will

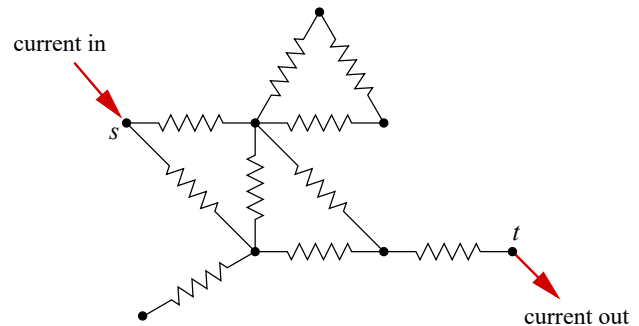


FIG. 5: An example of the type of resistor network considered here, in which a unit resistance is placed on each edge and unit current flows into and out of the source and sink vertices.

use the absolute magnitude of the current flow as our betweenness score for each source/sink pair.

The current flows in the network are governed by Kirchhoff's laws. To solve them we proceed as follows for each separate component of the graph. Let V_i be the voltage at vertex i , measured relative to any convenient point. Then for all i we have

$$\sum_j A_{ij}(V_i - V_j) = \delta_{is} - \delta_{it}, \quad (1)$$

where A_{ij} is the ij element of the adjacency matrix of the graph, i.e., $A_{ij} = 1$ if i and j are connected by an edge and $A_{ij} = 0$ otherwise. The left-hand side of Eq. (1) represents the net current flow out of vertex i along edges of the network, and the right-hand side represents the source and sink. Defining $k_i = \sum_j A_{ij}$, which is the vertex degree, and creating a diagonal matrix $\mathbf{D}$ with these degrees on the diagonal $D_{ii} = k_i$, this equation can be written in matrix form as $(\mathbf{D} - \mathbf{A}) \cdot \mathbf{V} = \mathbf{s}$, where the source vector $\mathbf{s}$ has components

$$s_i = \begin{cases} +1 & \text{for } i = s \\ -1 & \text{for } i = t \\ 0 & \text{otherwise.} \end{cases} \quad (2)$$

We cannot directly invert the matrix $\mathbf{D} - \mathbf{A}$ to get the voltage vector $\mathbf{V}$, because the matrix (which is just the graph Laplacian) is singular. This is equivalent to saying that there is one undetermined degree of freedom corresponding to the choice of reference potential for measuring the voltages. We can add any constant to a solution for the vertex voltages and get another solution—only the voltage differences matter. In choosing the reference potential, we fix this degree of freedom, leaving only $n - 1$ more to be determined. In mathematical terms, once any $n - 1$ of the equations in our matrix formulation are satisfied, the remaining one is also automatically satisfied so long as current is conserved in the network as a whole, i.e., so long as $\sum_i s_i = 0$, which is clearly true in this case.

Choosing any vertex v to be the reference point, therefore, we remove the row and column corresponding to

that vertex from $\mathbf{D}$ and $\mathbf{A}$ before inverting. Denoting the resulting $(n-1) \times (n-1)$ matrices $\mathbf{D}_v$ and $\mathbf{A}_v$, we can then write

$$\mathbf{V} = (\mathbf{D}_v - \mathbf{A}_v)^{-1} \cdot \mathbf{s}. \quad (3)$$

Calculation of the currents in the network thus involves inverting $\mathbf{D}_v - \mathbf{A}_v$ once for any convenient choice of v , and taking the differences of pairs of columns to get the voltage vector $\mathbf{V}$ for each possible source/sink pair. (The voltage for the one missing vertex v is always zero, by hypothesis.) The absolute magnitudes of the differences of voltages along each edge give us betweenness scores for the given source and sink. Summing over all sources and sinks, we then get our complete betweenness score.

The matrix inversion takes time $O(n^3)$ in the worst case, while the subsequent calculation of betweennesses takes time $O(mn^2)$, where as before m is the number of edges and n the number of vertices in the graph. Thus, the entire community structure algorithm, including the recalculation step, will take $O((n+m)mn^2)$ time to complete, or $O(n^4)$ on a sparse graph. Although, as we will see, the algorithm is good at finding community structure, this poor performance makes it practical only for smaller graphs; a few hundreds of vertices is the most that we have been able to do. It is for this reason that we recommend using the shortest-path betweenness algorithm in most cases, which gives results about as good or better with considerably less effort.

C. Random walks

The random-walk betweenness described in Sec. II requires us to calculate how often on average random walks starting at vertex s will pass down a particular edge from vertex v to vertex w (or *vice versa*) before finding their way to a given target vertex t . To calculate this quantity we proceed as follows for each separate component of the graph.

As before, let A_{ij} be an element of the adjacency matrix such that $A_{ij} = 1$ if vertices i and j are connected by an edge and $A_{ij} = 0$ otherwise. Consider a random walk that on each step decides uniformly between the neighbors of the current vertex j and takes a step to one of them. The number of neighbors is just the degree of the vertex $k_j = \sum_i A_{ij}$, and the probability for the transition from j to i is A_{ij}/k_j , which we can regard as an element of the matrix $\mathbf{M} = \mathbf{A} \cdot \mathbf{D}^{-1}$, where $\mathbf{D}$ is the diagonal matrix with $D_{ii} = k_i$.

We are interested in walks that terminate when they reach the target t , so that t is an absorbing state. The most convenient way to represent this is just to remove entirely the vertex t from the graph, so that no walk ever reaches any other vertex from t . Thus let $\mathbf{M}_t = \mathbf{A}_t \cdot \mathbf{D}_t^{-1}$ be the matrix $\mathbf{M}$ with the t th row and column removed (and similarly for $\mathbf{A}_t$ and $\mathbf{D}_t$).

Now the probability that a walk starts at s , takes n steps, and ends up at some other vertex (not t), is given

by the is element of $\mathbf{M}_t^n$, which we denote $[\mathbf{M}_t^n]_{is}$. In particular, walks end up at v and w with probabilities $[\mathbf{M}_t^n]_{vs}$ and $[\mathbf{M}_t^n]_{ws}$, and of those a fraction $1/k_v$ and $1/k_w$ respectively then pass along the edge (v, w) in one direction or the other. (Note that they may also have passed along this edge an arbitrary number of times before reaching this point.) Summing over all n , the mean number of times that a walk of any length traverses the edge from v to w is $k_v^{-1}[(\mathbf{I} - \mathbf{M}_t)^{-1}]_{vs}$, and similarly for walks that go from w to v .

To highlight the similarity with the current-flow betweenness of Sec. III B, let us denote these two numbers V_v and V_w respectively. Then we can write

$$\mathbf{V} = \mathbf{D}^{-1} \cdot (\mathbf{I} - \mathbf{M}_t)^{-1} \cdot \mathbf{s} = (\mathbf{D}_t - \mathbf{A}_t)^{-1} \cdot \mathbf{s}, \quad (4)$$

where the source vector $\mathbf{s}$ is the vector whose components are all 0 except for a single 1 in the position corresponding to the source vertex s .

Now we define our random-walk betweenness for the edge (v, w) to be the absolute value of the *difference* of the two probabilities V_v and V_w , i.e., the net number of times the walk passes along the edge in one direction. This seems a natural definition—it makes little sense to accord an edge high betweenness simply because a walk went back and forth along it many times. It is the difference between the numbers of times the edge is traversed in either direction that matters [48].

But now we see that this method is very similar to the resistor network calculation of Sec. III B. In that calculation we also evaluated $(\mathbf{D}_t - \mathbf{A}_t)^{-1} \cdot \mathbf{s}$ for a suitable source vector and then took differences of the resulting numbers. The only difference is that in the current-flow calculation we had a sink term in $\mathbf{s}$ as well as a source. Purely for the purposes of mathematical convenience, we can add such a sink in the present case at the target vertex t —this makes no difference to the solution for $\mathbf{V}$ since the t th row has been removed from the equations anyway. By doing this, however, we turn the equations into precisely the form of the current-flow calculation, and hence it becomes clear that the two measures are numerically identical, although their derivation is quite different. (It also immediately follows that we can remove any row or column and still get the same answer—it doesn't have to be row and column t , although physically this choice makes the most sense.)

IV. QUANTIFYING THE STRENGTH OF COMMUNITY STRUCTURE

As we show in Sec. V, our community structure algorithms do an excellent job of recovering known communities both in artificially generated random networks and in real-world examples. However, in practical situations the algorithms will normally be used on networks for which the communities are not known ahead of time. This raises a new problem: how do we know when the communities found by the algorithm are good ones? Our algo-

gorithms always produce *some* division of the network into communities, even in completely random networks that have no meaningful community structure, so it would be useful to have some way of saying how good the structure found is. Furthermore, the algorithms' output is in the form of a dendrogram which represents an entire nested hierarchy of possible community divisions for the network. We would like to know which of these divisions are the best ones for a given network—where we should cut the dendrogram to get a sensible division of the network.

To answer these questions we now define a measure of the quality of a particular division of a network, which we call the *modularity*. This measure is based on a previous measure of assortative mixing proposed by Newman [33]. Consider a particular division of a network into k communities. Let us define a $k \times k$ symmetric matrix $\mathbf{e}$ whose element e_{ij} is the fraction of all edges in the network that link vertices in community i to vertices in community j [49]. (Here we consider all edges in the original network—even after edges have been removed by the community structure algorithm our modularity measure is calculated using the full network.)

The trace of this matrix $\text{Tr } \mathbf{e} = \sum_i e_{ii}$ gives the fraction of edges in the network that connect vertices in the same community, and clearly a good division into communities should have a high value of this trace. The trace on its own, however, is not a good indicator of the quality of the division since, for example, placing all vertices in a single community would give the maximal value of $\text{Tr } \mathbf{e} = 1$ while giving no information about community structure at all.

So we further define the row (or column) sums $a_i = \sum_j e_{ij}$, which represent the fraction of edges that connect to vertices in community i . In a network in which edges fall between vertices without regard for the communities they belong to, we would have $e_{ij} = a_i a_j$. Thus we can define a modularity measure by

$$Q = \sum_i (e_{ii} - a_i^2) = \text{Tr } \mathbf{e} - \|\mathbf{e}^2\|, \quad (5)$$

where $\|\mathbf{x}\|$ indicates the sum of the elements of the matrix $\mathbf{x}$. This quantity measures the fraction of the edges in the network that connect vertices of the same type (i.e., within-community edges) minus the expected value of the same quantity in a network with the same community divisions but random connections between the vertices. If the number of within-community edges is no better than random, we will get $Q = 0$. Values approaching $Q = 1$, which is the maximum, indicate strong community structure [50]. In practice, values for such networks typically fall in the range from about 0.3 to 0.7. Higher values are rare.

The expected error on Q can be calculated by treating each edge in the network as an independent measurement of the contributions to the elements of the matrix $\mathbf{e}$. A simple jackknife procedure works well [33, 34].

Typically, we will calculate Q for each split of a net-

work into communities as we move down the dendrogram, and look for local peaks in its value, which indicate particularly satisfactory splits. Usually we find that there are only one or two such peaks and, as we will show in the next section, in cases where the community structure is known beforehand by some means we find that the positions of these peaks correspond closely to the expected divisions. The height of a peak is a measure of the strength of the community division.

V. APPLICATIONS

In this section we give a number of applications of our algorithms to particular problems, illustrating their operation, and their use in understanding the structure of complex networks.

A. Tests on computer-generated networks

First, as a controlled test of how well our algorithms perform, we have generated networks with known community structure, to see if the algorithms can recognize and extract this structure.

We have generated a large number of graphs with $n = 128$ vertices, divided into four communities of 32 vertices each. Edges were placed independently at random between vertex pairs with probability p_{in} for an edge to fall between vertices in the same community and p_{out} to fall between vertices in different communities. The values of p_{in} and p_{out} were chosen to make the expected degree of each vertex equal to 16. In Fig. 6 we show a typical dendrogram from the analysis of such a graph using the shortest-path betweenness version of our algorithm. (In fact, for the sake of clarity, the figure is for a 64-node version of the graph.) Results for the random walk version are similar. At the top of the figure we also show the modularity, Eq. (5), for the same calculation, plotted as a function of position in the dendrogram. That is, the graph is aligned with the dendrogram so that one can read off modularity values for different divisions of the network directly. As we can see, the modularity has a single clear peak at the point where the network breaks into four communities, as we would expect. The peak value is around 0.5, which is typical.

In Fig. 7 we show the fraction of vertices in our computer-generated network sample classified correctly into the four communities by our algorithms, as a function of the mean number z_{out} of edges from each vertex to vertices in other communities. As the figure shows, both the shortest-path and random-walk versions of the algorithm perform excellently, with more than 90% of all vertices classified correctly from $z_{\text{out}} = 0$ all the way to around $z_{\text{out}} = 6$. Only for $z_{\text{out}} \gtrsim 6$ does the classification begin to deteriorate markedly. In other words, our algorithm correctly identifies the community structure in the network almost all the way to the point $z_{\text{out}} = 8$ at



FIG. 6: Plot of the modularity and dendrogram for a 64-vertex random community-structured graph generated as described in the text with, in this case, $z_{\text{in}} = 6$ and $z_{\text{out}} = 2$. The shapes on the right denote the four communities in the graph and as we can see, the peak in the modularity (dotted line) corresponds to a perfect identification of the communities.

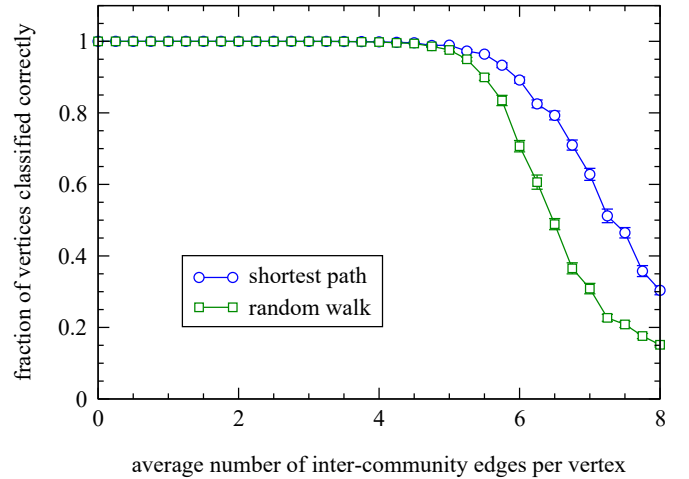


FIG. 7: The fraction of vertices correctly identified by our algorithms in the computer-generated graphs described in the text. The two curves show results for the edge betweenness (circles) and random walk (squares) versions of the algorithm as a function of the number of edges vertices have to others outside their own community. The point $z_{\text{out}} = 8$ at the rightmost edge of the plot represents the point at which the graphs—in this example—have as many connections outside their own community as inside it. Each point is an average over 100 graphs.

which each vertex has on average the same number of connections to vertices outside its community as it does to those inside.

The shortest-path version of the algorithm does however perform noticeably better than the random-walk version, especially for the more difficult cases where z_{out} is large. Given that the random-walk algorithm is also more computationally demanding, there seems little reason to use it rather than the shortest-path algorithm, and hence, as discussed previously, we recommend the latter for most applications. (To be fair, the random-walk algorithm does slightly out-perform the shortest-path algorithm in the example addressed in the following section, although, being only a single case, it is hard to know whether this is significant.)

B. Zachary's karate club network

We now turn to applications of our methods to real-world network data. Our first such example is taken from one of the classic studies in social network analysis. Over the course of two years in the early 1970s, Wayne Zachary observed social interactions between the members of a karate club at an American university [35]. He constructed networks of ties between members of the club based on their social interactions both within the club and away from it. By chance, a dispute arose during the course of his study between the club's adminis-

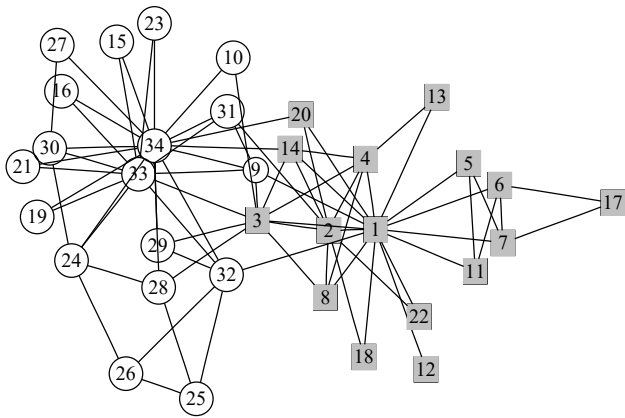


FIG. 8: The network of friendships between individuals in the karate club study of Zachary [35]. The administrator and the instructor are represented by nodes 1 and 33 respectively. Shaded squares represent individuals to who ended up aligning with the club’s administrator after the fission of the club, open circles those who aligned with the instructor.

trator and its principal karate teacher over whether to raise club fees, and as a result the club eventually split in two, forming two smaller clubs, centered around the administrator and the teacher.

In Fig. 8 we show a consensus network structure extracted from Zachary’s observations before the split. Feeding this network into our algorithms we find the results shown in Fig. 9. In the left-most two panels we show the dendrograms generated by the shortest-path and random-walk versions of our algorithm, along with the modularity measures for the same. As we see, both algorithms give reasonably high values for the modularity when the network is split into two communities—around 0.4 in each case—indicating that there is a strong natural division at this level. What’s more, the divisions in question correspond almost perfectly to the actual divisions in the club revealed by which group each club member joined after the club split up. (The shapes of the vertices representing the two factions are the same as those of Fig. 8.) Only one vertex, vertex 3, is misclassified by the shortest-path version of the method, and none are misclassified by the random-walk version—the latter gets a perfect score on this test. (On the other hand, the two-community split fails to produce a local maximum in the modularity for the random-walk method, unlike the shortest-path method for which there is a local maximum precisely at this point.)

In the last panel of Fig. 9 we show the dendrogram and modularity for an algorithm based on shortest-path betweenness but without the crucial recalculation step discussed in Sec. II. As the figure shows, without this step, the algorithm fails to find the division of the network into the two known groups. Furthermore, the modularity doesn’t reach nearly such high values as in the first two panels, indicating that the divisions suggested

are much poorer than in the cases with the recalculation.

C. Collaboration network

For our next example, we look at a collaboration network of scientists. Figure 10a shows the largest component of a network of collaborations between physicists who conduct research on networks. (The authors of the present paper, for instance, are among the nodes in this network.) This network (which appeared previously in Ref. 36) was constructed by taking names of authors appearing in the lengthy bibliography of Ref. 4 and cross-referencing with the Physics E-print Archive at arxiv.org, specifically the condensed matter section of the archive where, for historical reasons, most papers on networks have appeared. Authors appearing in both were added to the network as vertices, and edges between them indicate coauthorship of one or more papers appearing in the archive. Thus the collaborative ties represented in the figure are not limited to papers on topics concerning networks—we were interested primarily in whether people know one another, and collaboration on any topic is a reasonable indicator of acquaintance.

The network as presented in Fig. 10a is difficult to interpret. Given the names of the scientists, a knowledgeable reader with too much time on their hands could, no doubt, pick out known groupings, for instance at particular institutions, from the general confusion. But were this a network about which we had no *a priori* knowledge, we would be hard pressed to understand its underlying structure.

Applying the shortest-path version of our algorithm to this network we find that the modularity, Eq. (5), has a strong peak at 13 communities with a value of $Q = 0.72 \pm 0.02$. Extracting the communities from the corresponding dendrogram, we have indicated them with colors in Fig. 10b. The knowledgeable reader will again be able to discern known groups of scientists in this rendering, and more easily now with the help of the colors. Still, however, the structure of the network as a whole and the of the interactions between groups is quite unclear.

In Fig. 10c we have reduced the network to *only* the groups. In this panel, we have drawn each group as a circle, with size varying roughly with the number of individuals in the group. The lines between groups indicate collaborations between group members, with the thickness of the lines varying in proportion to the number of pairs of scientists who have collaborated. Now the overall structure of the network becomes easy to see. The network is centered around the middle group shown in cyan (which consists of researchers primarily in southern Europe), with a knot of inter-community collaborations going on between the groups on the lower right of the picture (mostly Boston University physicists and their intellectual descendants). Other groups (including the authors’ own) are arranged in various attitudes further

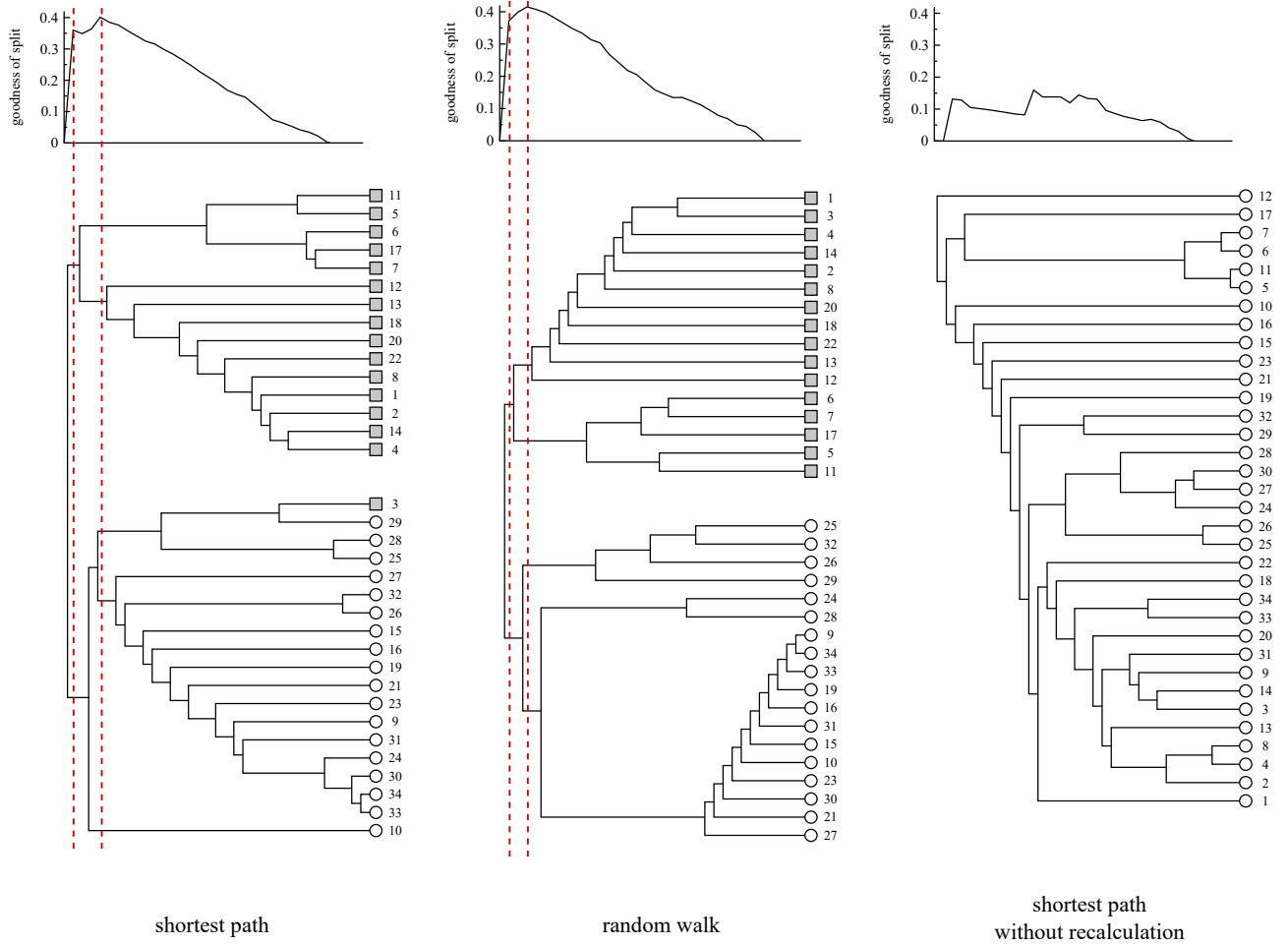


FIG. 9: Community structure in the karate club network. Left: the dendrogram extracted by the shortest-path betweenness version of our method, and the resulting modularity. The modularity has two maxima (dotted lines) corresponding to splits into two communities (which match closely the real-world split of the club, as denoted by the shapes of the vertices) and five communities (though one of those five contains only one individual). Only one individual, number 3, is incorrectly classified. Center: the dendrogram for the random walk version of our method. This version classifies all 34 vertices correctly into the factions that they actually split into (first dotted line), although the split into four communities gets a higher modularity score (second dotted line). Right: the dendrogram for the shortest-path algorithm without recalculation of betweennesses after each edge removal. This version of the calculation fails to find the split into the two factions.

out.

One of the problems created by the sudden availability in recent years of large network data sets has been our lack of tools for visualizing their structure [4]. In the early days of network analysis, particularly in the social sciences, it was usually enough simply to draw a picture of a network to see what was going on. Networks in those days had ten or twenty nodes, not 140 as here, or several billion as in the world wide web. We believe that methods like the one presented here, of using community structure algorithms to make a meaningful “coarse graining” of a network, thereby reducing its level of complexity to one that can be interpreted readily by the human eye, will be invaluable in helping us to understand the large-scale structure of these new network data.

D. Other examples

In this section, we briefly describe example applications of our methods to three further networks. The first is a non-human social network, a network of dolphins, the second a network of fictional characters, and the third not a social network at all, but a network of web pages and the links between them.

In Fig. 11 we show the social network of a community of 62 bottlenose dolphins living in Doubtful Sound, New Zealand. The network was compiled by Lusseau [37] from seven years of field studies of the dolphins, with ties between dolphin pairs being established by observation of statistically significant frequent association. The network splits naturally into two large groups, represented by the

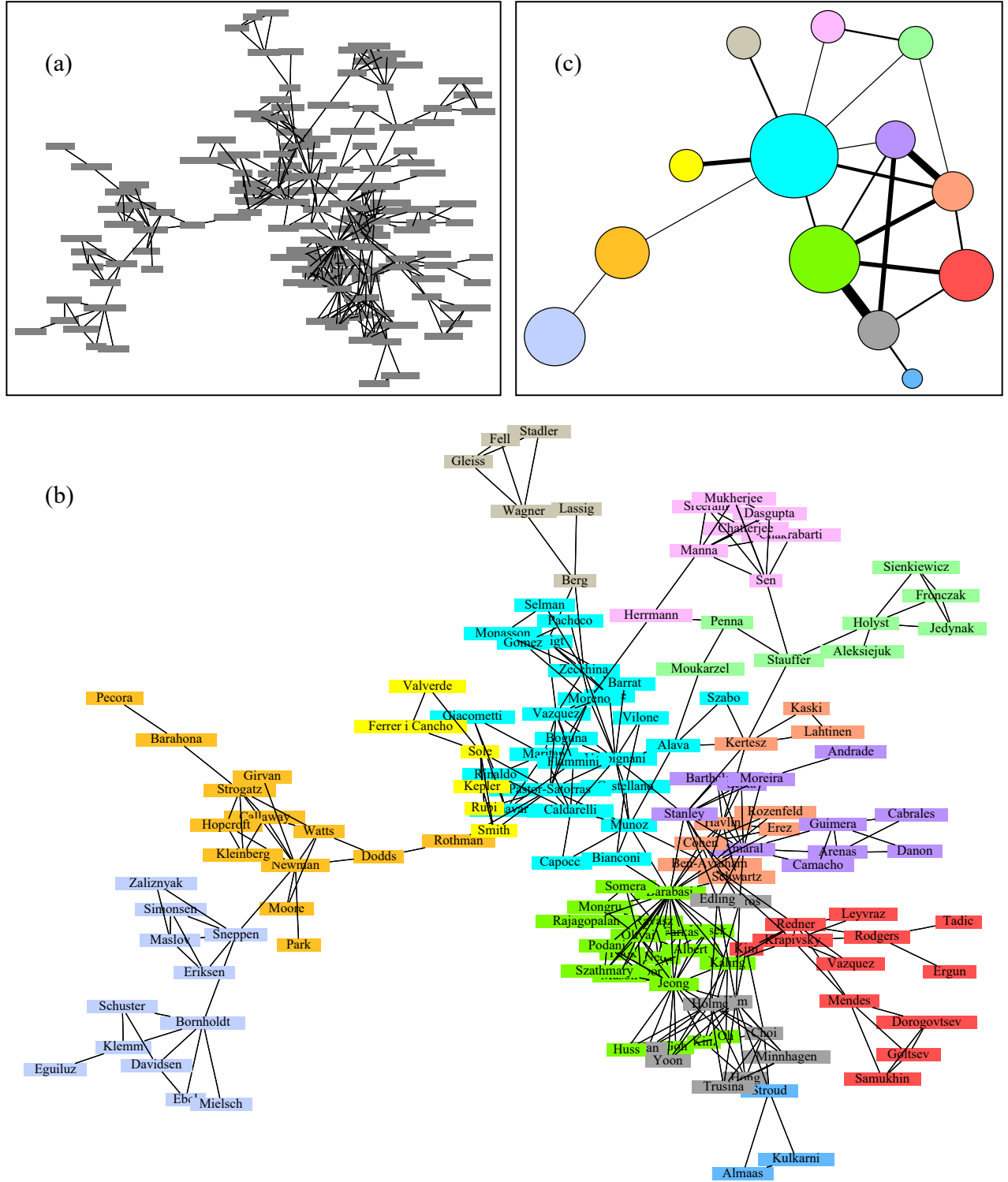


FIG. 10: Illustration of the use of the community structure algorithm to make sense of a complex network. (a) The initial network is a network of coauthorships between physicists who have published on topics related to networks. The figure shows only the largest component of the network, which contains 145 scientists. There are 90 more scientists in smaller components, which are not shown. (b) Application of the shortest-path betweenness version of the community structure algorithm produces the communities shown by the colors. (c) A coarse-graining of the network in which each community is represented by a single node, with edges representing collaborations between communities. The thickness of the edges is proportional to the number of pairs of collaborators between communities. Clearly panel (c) reveals much that is not easily seen in the original network of panel (a).

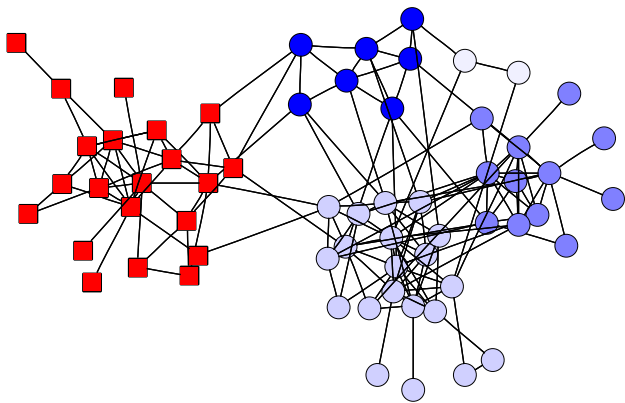


FIG. 11: Community structure in the bottlenose dolphins of Doubtful Sound [37, 38], extracted using the shortest-path version of our algorithm. The squares and circles denote the primary split of the network into two groups, and the circles are further subdivided into four smaller groups denoted by the different shades of vertices. The modularity for the split is $Q = 0.52$. The network has been drawn with longer edges between vertices in different communities than between those in the same community, to make the community groupings clearer. The same is also true of Figs. 12 and 13.

circles and squares in the figure, and the larger of the two also splits into four smaller subgroups, represented by the different shades. The modularity is $Q = 0.38 \pm 0.08$ for the split into two groups, and peaks at 0.52 ± 0.03 when the subgroup splitting is included also.

The split into two groups appears to correspond to a known division of the dolphin community [38]. Lusseau reports that for a period of about two years during observation of the dolphins they separated into two groups along the lines found by our analysis, apparently because of the disappearance of individuals on the boundary between the groups. When some of these individuals later reappeared, the two halves of the network joined together once more. As Lusseau points out, developments of this kind illustrate that the dolphin network is not merely a scientific curiosity but, like human social networks, is closely tied to the evolution of the community. The subgroupings within the larger half of the network also seem to correspond to real divisions among the animals: the largest subgroup consists almost entirely of females and the others almost entirely of males, and it is conjectured that the split between the male groups is governed by matrilineage (D. Lusseau, personal communication).

Figure 12 shows the community structure of the network of interactions between major characters in Victor Hugo’s sprawling novel of crime and redemption in post-restoration France, *Les Misérables*. Using the list of character appearances by scene compiled by Knuth [39], the network was constructed in which the vertices represent characters and an edge between two vertices represents co-appearance of the corresponding characters in one or more scenes. The optimal community split of the result-

ing graph has a strong modularity of $Q = 0.54 \pm 0.02$, and gives 11 communities as shown in the figure. The communities clearly reflect the subplot structure of the book: unsurprisingly, the protagonist Jean Valjean and his nemesis, the police officer Javert, are central to the network and form the hubs of communities composed of their respective adherents. Other subplots centered on Marius, Cosette, Fantine, and the bishop Myriel are also picked out.

Finally, as an example of the application of our method to a non-social network, we have looked at a web graph—a network in which the vertices and edges represent web pages and the links between them. The graph in question represents 180 pages from the web site of a large corporation [51]. Figure 13 shows the network and the communities found in it by the shortest-path version of our algorithm. This network has one of the strongest modularity values of the examples studied here, at $Q = 0.65 \pm 0.02$. The links between web pages are directed, as indicated by the arrows in the figure, but, as discussed in Sec. III A, for the purposes of finding the communities we ignore direction and treat the network as undirected.

Certainly it might be useful to know the communities in a web network; an algorithm that can pick out communities could reveal which pages cover related topics or the social structure of links between pages maintained by different individuals. Ideas along these lines have been pursued by, for example, Flake *et al.* [40] and Adamic and Adar [41].

VI. CONCLUSIONS

In this paper we have described a new class of algorithms for performing network clustering, the task of extracting the natural community structure from networks of vertices and edges. This is a problem long studied in computer science, applied mathematics, and the social sciences, but it has lacked a satisfactory solution. We believe the methods described here give such a solution. They are simple, intuitive, and demonstrably give excellent results on networks for which we know the community structure ahead of time. Our methods are defined by two crucial features. First, we use a “divisive” technique which iteratively removes edges from the network, thereby breaking it up in communities. The edges to be removed are identified by using one of a set of edge betweenness measures, of which the simplest is a generalization to edges of the standard shortest-path betweenness of Freeman. Second, our algorithms include a recalculation step in which betweenness scores are re-evaluated after the removal of every edge. This step, which was missing from previous algorithms, turns out to be of primary importance to the success of ours. Without it, the algorithms fail miserably at even the simplest clustering tasks.

We have demonstrated the efficacy and utility of our methods with a number of examples. We have shown

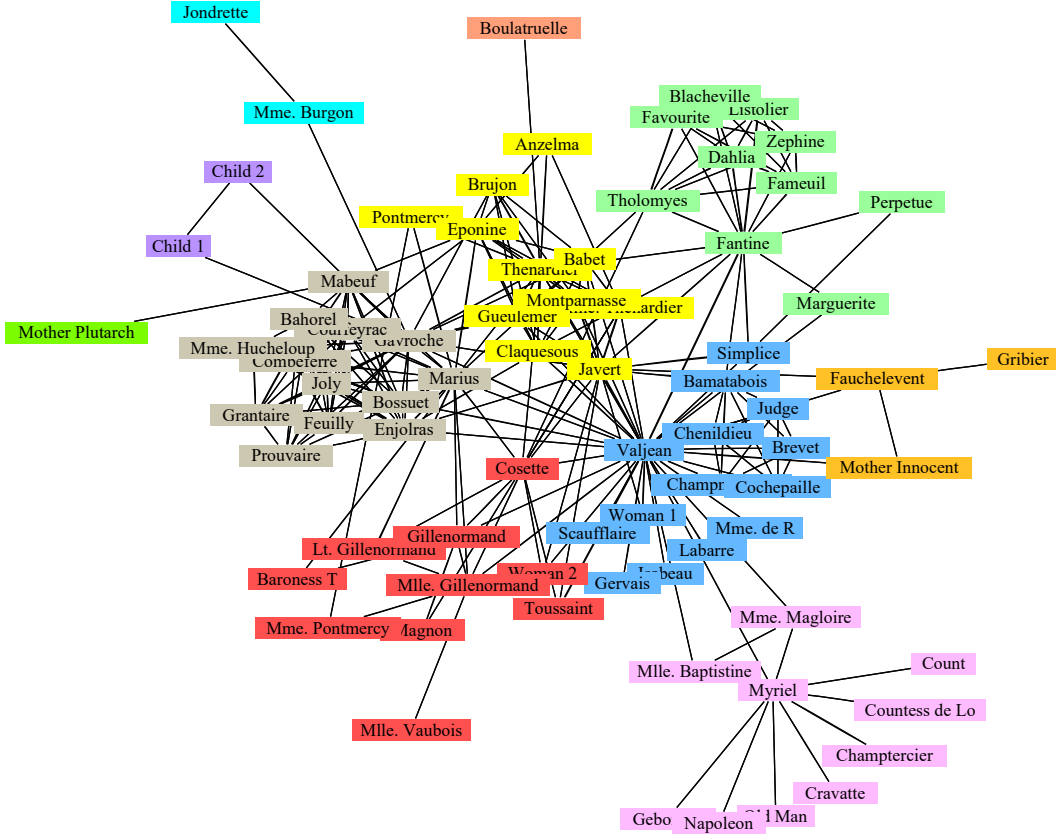


FIG. 12: The network of interactions between major characters in the novel *Les Misérables* by Victor Hugo. The greatest modularity achieved in the shortest-path version of our algorithm is $Q = 0.54$ and corresponds to the 11 communities represented by the colors.

that our algorithms can reliably and sensitively extract community structure from artificially generated networks with known communities. We have also applied them to real-world networks with known community structure and again they extract that structure without difficulty. And we have given examples of how our algorithms can be used to analyze networks whose structure is otherwise difficult to comprehend. The networks studied include a collaboration network of scientists, in which our methods allow us to generate schematic depictions of the overall structure of the network and collaborations taking place within and between communities, other social networks of people and of animals, and a network of links between pages on a corporate web site.

The primary remaining difficulty with our algorithms is the relatively high computational demands they make. The fastest of them, the one based on shortest-path betweenness, operates in $O(n^3)$ time on a sparse graph, which makes it usable for networks up to about 10 000 vertices, but for larger systems it becomes intractable. Although the ever-improving speed of computers will certainly raise this limit in coming years, it would be more satisfactory if a faster version of the method could be discovered. One possibility is parallelization: the between-

ness calculation involves a sum over source vertices and the elements of that sum can be distributed over different processors, making the calculation trivially parallelizable on a distributed-memory machine. However, a better approach would be to find some improvement in the algorithm itself to decrease its computational complexity.

Since the publication of our first paper on this topic [25], several other authors have made use of the shortest-path version of our algorithm. Holme *et al.* [42] have applied it to a number of metabolic networks for different organisms, finding communities that correspond to functional units within the networks, while Wilkinson and Huberman [43] have applied it to a network of relations between genes, as established by co-occurrence of names of genes in published research articles. An interesting application to social networks is the study by Gleiser and Danon [44] of the collaboration network of early jazz musicians. They found, among other things, that the network split into two communities along lines of race, black musicians in one group, white musicians in the other. Guimerà *et al.* [45] have applied the method to a network of email messages passing between users at a university, and found communities that reflect both formal and informal levels of organization. Tyler *et al.* [46] have

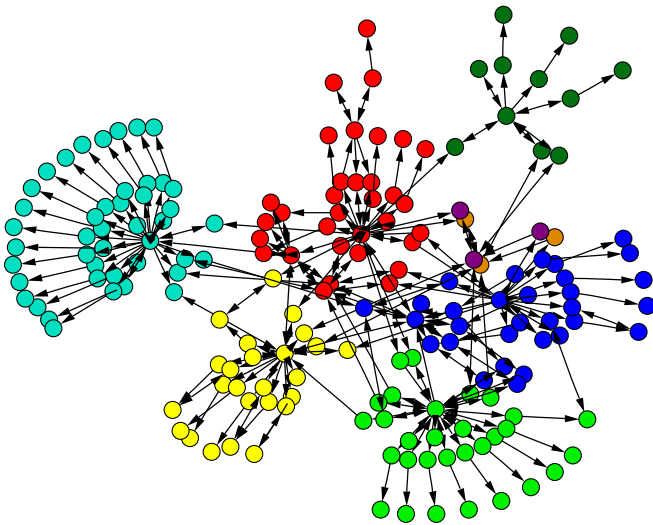


FIG. 13: Pages on a web site and the hyperlinks between them. The colors denote the optimal division into communities found by the shortest-path version of our algorithm.

also applied the algorithm to an email network, in their case at a large company, finding that the resulting communities correspond closely to organizational units. The latter work is interesting also in that it suggests a method for improving the speed of the algorithm: Tyler *et al.* calculate betweenness for only a subset, randomly chosen,

of possible source vertices in the network, rather than summing over all sources. The size of the subset is decided on the fly, by sampling source vertices until the betweenness of at least one edge in the network exceeds a predetermined threshold. This technique reduces the running time of the calculation considerably, although the resulting estimate of betweenness necessarily suffers from the statistical fluctuations inherent in random sampling methods. This idea, or a variation of it, might provide a solution to the problems mentioned above of the high computational demands of our algorithms.

We are of course delighted to see our methods applied to such a variety of problems. Combined with the new algorithms and measures described in this paper, we hope to see many more applications in the future.

Acknowledgments

The authors thank Steven Borgatti, Ulrik Brandes, Linton Freeman, David Lusseau, Mason Porter, and Douglas White for useful comments. Thanks also to Oliver Boisseau, Patti Haase, David Lusseau, and Karsten Schneider for providing the data for the dolphin network and to Douglas White for the karate club data. This work was funded in part by the National Science Foundation under grant number DMS-0234188 and by the Santa Fe Institute.

-
- [1] S. H. Strogatz, Exploring complex networks. *Nature* **410**, 268–276 (2001).
 - [2] R. Albert and A.-L. Barabási, Statistical mechanics of complex networks. *Rev. Mod. Phys.* **74**, 47–97 (2002).
 - [3] S. N. Dorogovtsev and J. F. F. Mendes, *Evolution of Networks: From Biological Nets to the Internet and WWW*. Oxford University Press, Oxford (2003).
 - [4] M. E. J. Newman, The structure and function of complex networks. *SIAM Review* **45**, 167–256 (2003).
 - [5] M. Faloutsos, P. Faloutsos, and C. Faloutsos, On power-law relationships of the internet topology. *Computer Communications Review* **29**, 251–262 (1999).
 - [6] R. Albert, H. Jeong, and A.-L. Barabási, Diameter of the world-wide web. *Nature* **401**, 130–131 (1999).
 - [7] A. Broder, R. Kumar, F. Maghoul, P. Raghavan, S. Rajagopalan, R. Stata, A. Tomkins, and J. Wiener, Graph structure in the web. *Computer Networks* **33**, 309–320 (2000).
 - [8] A. Kleczkowski and B. T. Grenfell, Mean-field-type equations for spread of epidemics: The ‘small world’ model. *Physica A* **274**, 355–360 (1999).
 - [9] C. Moore and M. E. J. Newman, Epidemics and percolation in small-world networks. *Phys. Rev. E* **61**, 5678–5682 (2000).
 - [10] R. Pastor-Satorras and A. Vespignani, Epidemic spreading in scale-free networks. *Phys. Rev. Lett.* **86**, 3200–3203 (2001).
 - [11] R. M. May and A. L. Lloyd, Infection dynamics on scale-free networks. *Phys. Rev. E* **64**, 066112 (2001).
 - [12] S. Redner, How popular is your paper? An empirical study of the citation distribution. *Eur. Phys. J. B* **4**, 131–134 (1998).
 - [13] M. E. J. Newman, The structure of scientific collaboration networks. *Proc. Natl. Acad. Sci. USA* **98**, 404–409 (2001).
 - [14] H. Jeong, B. Tombor, R. Albert, Z. N. Oltvai, and A.-L. Barabási, The large-scale organization of metabolic networks. *Nature* **407**, 651–654 (2000).
 - [15] A. Wagner and D. Fell, The small world inside large metabolic networks. *Proc. R. Soc. London B* **268**, 1803–1810 (2001).
 - [16] J. A. Dunne, R. J. Williams, and N. D. Martinez, Food-web structure and network theory: The role of connectance and size. *Proc. Natl. Acad. Sci. USA* **99**, 12917–12922 (2002).
 - [17] J. Camacho, R. Guimerà, and L. A. N. Amaral, Robust patterns in food web structure. *Phys. Rev. Lett.* **88**, 228102 (2002).
 - [18] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco (1979).
 - [19] J. Scott, *Social Network Analysis: A Handbook*. Sage Publications, London, 2nd edition (2000).
 - [20] B. W. Kernighan and S. Lin, A efficient heuristic procedure

- cedure for partitioning graphs. *Bell System Technical Journal* **49**, 291–307 (1970).
- [21] D. J. Watts and S. H. Strogatz, Collective dynamics of ‘small-world’ networks. *Nature* **393**, 440–442 (1998).
- [22] L. A. N. Amaral, A. Scala, M. Barthélemy, and H. E. Stanley, Classes of small-world networks. *Proc. Natl. Acad. Sci. USA* **97**, 11149–11152 (2000).
- [23] M. Marchiori and V. Latora, Harmony in the small-world. *Physica A* **285**, 539–546 (2000).
- [24] R. L. Breiger, S. A. Boorman, and P. Arabie, An algorithm for clustering relations data with applications to social network analysis and comparison with multidimensional scaling. *Journal of Mathematical Psychology* **12**, 328–383 (1975).
- [25] M. Girvan and M. E. J. Newman, Community structure in social and biological networks. *Proc. Natl. Acad. Sci. USA* **99**, 7821–7826 (2002).
- [26] J. M. Anthonisse, The rush in a directed graph. Technical Report BN 9/71, Stichting Mathematicsh Centrum, Amsterdam (1971).
- [27] L. Freeman, A set of measures of centrality based upon betweenness. *Sociometry* **40**, 35–41 (1977).
- [28] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Upper Saddle River, New Jersey (1993).
- [29] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. MIT Press, Cambridge, MA, 2nd edition (2001).
- [30] M. E. J. Newman, Scientific collaboration networks: II. Shortest paths, weighted networks, and centrality. *Phys. Rev. E* **64**, 016132 (2001).
- [31] U. Brandes, A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology* **25**, 163–177 (2001).
- [32] K.-I. Goh, B. Kahng, and D. Kim, Universal behavior of load distribution in scale-free networks. *Phys. Rev. Lett.* **87**, 278701 (2001).
- [33] M. E. J. Newman, Mixing patterns in networks. *Phys. Rev. E* **67**, 026126 (2003).
- [34] B. Efron, Computers and the theory of statistics: Thinking the unthinkable. *SIAM Review* **21**, 460–480 (1979).
- [35] W. W. Zachary, An information flow model for conflict and fission in small groups. *Journal of Anthropological Research* **33**, 452–473 (1977).
- [36] J. Park and M. E. J. Newman, The origin of degree correlations in the Internet and other networks. Preprint cond-mat/0303327 (2003).
- [37] D. Lusseau, The emergent properties of a dolphin social network. *Biology Letters, Proc. R. Soc. London B (suppl.)* (2003). DOI 10.1098/rsbl.2003.0057.
- [38] D. Lusseau, K. Schneider, O. J. Boisseau, P. Haase, E. Slooten, and S. M. Dawson, The bottlenose dolphin community of Doubtful Sound features a large proportion of long-lasting associations. Can geographic isolation explain this unique trait? *Behavioral Ecology and Sociobiology* (2003). DOI 10.1007/s00265-003-0651-y.
- [39] D. E. Knuth, *The Stanford GraphBase: A Platform for Combinatorial Computing*. Addison-Wesley, Reading, MA (1993).
- [40] G. W. Flake, S. R. Lawrence, C. L. Giles, and F. M. Coetzee, Self-organization and identification of Web communities. *IEEE Computer* **35**, 66–71 (2002).
- [41] L. A. Adamic and E. Adar, Friends and neighbors on the Web. *Social Networks* **25**, 211–230 (2003).
- [42] P. Holme, M. Huss, and H. Jeong, Subnetwork hierarchies of biochemical pathways. *Bioinformatics* **19**, 532–538 (2003).
- [43] D. Wilkinson and B. A. Huberman, Finding communities of related genes. Preprint cond-mat/0210147 (2002).
- [44] P. Gleiser and L. Danon, Community structure in jazz. Preprint cond-mat/0307434 (2003).
- [45] R. Guimerà, L. Danon, A. Díaz-Guilera, F. Giralt, and A. Arenas, Self-similar community structure in organisations. Preprint cond-mat/0211498 (2002).
- [46] J. R. Tyler, D. M. Wilkinson, and B. A. Huberman, Email as spectroscopy: Automated discovery of community structure within organizations. Preprint cond-mat/0303264 (2003).
- [47] Following the publication of that paper, the algorithm has been implemented in the commercial network analysis software package UCInet and in the open-source network library JUNG. (See <http://www.analytictech.com/> and <http://jung.sourceforge.net/> respectively.)
- [48] In fact we have tried counting each traversal separately, but this method gives extremely poor results, confirming our intuition that this would not be a good betweenness measure.
- [49] As discussed in [33], it is crucial to make sure each edge is counted only once in the matrix e_{ij} —the same edge should not appear both above and below the diagonal. Alternatively, an edge linking communities i and j can be split, half-and-half, between the ij and ji elements, which has the advantage of making the matrix symmetric. Either way, there are a number of factors of 2 in the calculation that must be watched carefully, lest they escape one’s attention and make mischief.
- [50] In Ref. 33, the measure was normalized by dividing by its value on a network with perfect mixing, so that we always get 1 for such a network. We find however that doing this in the present case masks some of the useful information to be gained from the value of Q , and hence that it is better to use the unnormalized measure. In general, this unnormalized measure will not reach a value of 1, even on a perfectly mixed network.
- [51] The graph is one of the test graphs from the graph drawing competition held in conjunction with the Symposium on Graph Drawing, Berkeley, California, September 18–20, 1996.